

INFORME TÉCNICO DE INGENIERÍA SPRINT 1: ARQUITECTURA BASE Y FUNDACIÓN

Consolidación del Modelo Relacional, Motor de Migraciones DDL,
Autenticación Criptográfica JWT y Matriz de Autorización RBAC

Producto:	Sistema Wayka MVP (Versión 1.0.0-sprint1)
Fase / Sprint:	Sprint 1 (Semanas 1 y 2 – Fundación Técnica)
Período de Ejecución:	01 de Septiembre al 14 de Septiembre de 2026
Asignatura:	SHC170 – Práctica Laboral
Estado de Ejecución:	Completada

Índice

1. Resumen Ejecutivo 2

1.1. Hitos y Logros Principales 2

2. Arquitectura de Software y Modelo de Datos 2

2.1. Estrategia de Modelado y Persistencia Relacional 3

2.2. Estrategia de Migraciones y Auditoría de Datos 3

2.3. Diagrama Entidad-Relación (ERD) en TikZ 3

3. Sistema de Seguridad, Autenticación y Autorización 4

3.1. Ciclo de Vida y Flujo Criptográfico del Token JWT 4

3.2. Matriz de Permisos Basada en Roles (RBAC) 5

4. Especificación de la API (Endpoints Desarrollados) 6

4.1. Catálogo de Rutas HTTP del Sprint 1 6

4.2. Ejemplos de Cargas Útiles (Payloads JSON) 7

4.2.1. 1. Inicio de Sesión Exitoso (POST /api/auth/login) 7

4.2.2. 2. Conmutación Dinámica de Rol (POST /api/auth/switch-role) 8

5. Pruebas y Validación (Quality Assurance) 8

5.1. Resultados del Banco de Pruebas de Autenticación (testAuth.js) 8

5.2. Resultados de Integridad del Modelo de Datos (testDataModel.js) 9

6. Estado de Avance y Próximos Pasos 9

6.1. Tabla de Entregables Planificados vs. Ejecutados 10

1. Resumen Ejecutivo

El presente documento constituye el Informe Técnico Exhaustivo correspondiente a la culminación del **Sprint 1 (Fundación)** del proyecto **Sistema Wayka MVP (Sistema de Gestión Documental y Workflow Institucional)**, implementado para modernizar la administración y trazabilidad de expedientes y correspondencias en el Gobierno Autónomo Municipal de Sucre.

Durante este primer ciclo de dos semanas (01 al 14 de Septiembre de 2026), se establecieron con éxito los cimientos arquitectónicos, estructurales y de seguridad de la plataforma, garantizando una base desacoplada, escalable y con alta cohesión técnica.

1.1. Hitos y Logros Principales

- **Capa de Persistencia y Motor DDL:** Diseño e implementación del esquema relacional unificado en MySQL 8.0+ mediante un motor automatizado de migraciones versionadas (`migrator.js`) y 11 scripts incrementales ordenados cronológicamente (`001_create_personas.sql` a `011_create_parametros.sql`).
- **Subsistema Criptográfico y Autenticación JWT:** Construcción de los servicios de emisión, validación y renovación de tokens criptográficos *JSON Web Token* (JWT) con firma HMAC-SHA256, incorporación de secretos distribuidos y hashing de contraseñas de alta seguridad mediante `bcrypt` (10 rondas de salt).
- **Autorización Multi-Rol y Modelo RBAC:** Estructuración de la matriz de control de acceso basada en roles (*Role-Based Access Control* – RBAC) para los cuatro roles esenciales del MVP: *Administrador de Sistema*, *Administrador de Wayka*, *Ventanilla Única* y *Funcionario*, soportando pertenencia simultánea a múltiples oficinas orgánicas.
- **Capa de Controladores y Servicios REST API:** Despliegue de los endpoints de autenticación, control de perfiles, conmutación dinámica de contexto operativo (`switch-role`) y modelos de acceso a datos (DAO) para Personas, Usuarios, Ubicaciones Orgánicas y Asignaciones.
- **Garantía de Calidad y Pruebas Automatizadas:** Verificación exhaustiva con suites de pruebas unitarias y de integración end-to-end (`testAuth.js` y `testDataModel.js`), logrando una tasa de éxito del 100 % (17/17 aserciones críticas aprobadas sin regresiones).

2. Arquitectura de Software y Modelo de Datos

El Sistema Wayka está diseñado bajo un enfoque de **Arquitectura en Capas Limpia (Clean Layered Architecture)** y desacoplamiento API-First. La capa de persistencia se apoya en un motor relacional MySQL 8.0+ con motor de almacenamiento InnoDB, garantizando cumplimiento riguroso de las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

2.1. Estrategia de Modelado y Persistencia Relacional

La arquitectura de datos del Sprint 1 resuelve tres requerimientos críticos de la gestión pública:

1. **Unificación de la Identidad Civil e Institucional:** Separación explícita entre la identidad física de un ciudadano o servidor público (**personas**) y su cuenta digital de acceso (**usuarios**), permitiendo que una persona mantenga su historial aun cuando cambie de cargo o dependencia.
2. **Organigrama Jerárquico Recursivo:** La tabla `ubicaciones_org` implementa una estructura de árbol adyacente (*Adjacency List*) mediante el campo autorreferencial `padre_id`, calculando automáticamente los niveles administrativos ($N + 1$).
3. **Modelo Multi-Rol Contextualizado:** A través de la tabla asociativa `usuario_rols`, un usuario puede ejercer diferentes funciones según la oficina u órgano municipal en el que esté asignado, gobernado por restricciones de unicidad compuesta (`uk_usuario_rol_ubicacion`) y fechas de caducidad.

2.2. Estrategia de Migraciones y Auditoría de Datos

Para evitar discrepancias entre entornos de desarrollo, pruebas y producción, se prescindió del modelado manual ad-hoc, adoptando un subsistema de migraciones idempotente gobernado por la tabla de control `_migrations`:

- **Integridad Referencial Estricta:** Todas las llaves foráneas se definieron con la cláusula `ON DELETE RESTRICT`, bloqueando eliminaciones en cascada accidentales que pudieran comprometer la trazabilidad de expedientes oficiales.
- **Borrado Lógico (*Soft Delete*):** Las entidades transaccionales y de catálogo implementan el flag booleano `activo` `DEFAULT 1`. La capa de lógica de negocio valida dependencias antes de desactivar cualquier registro (ej. una persona no puede desactivarse si cuenta con un usuario activo).
- **Auditoría Temporal Nativa:** Inclusión sistemática de marcas temporales `created_at` `TIMESTAMP DEFAULT CURRENT_TIMESTAMP` y `updated_at` `TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`.

2.3. Diagrama Entidad-Relación (ERD) en TikZ

La Figura 1 ilustra la topología relacional de las entidades fundacionales construidas durante el Sprint 1.

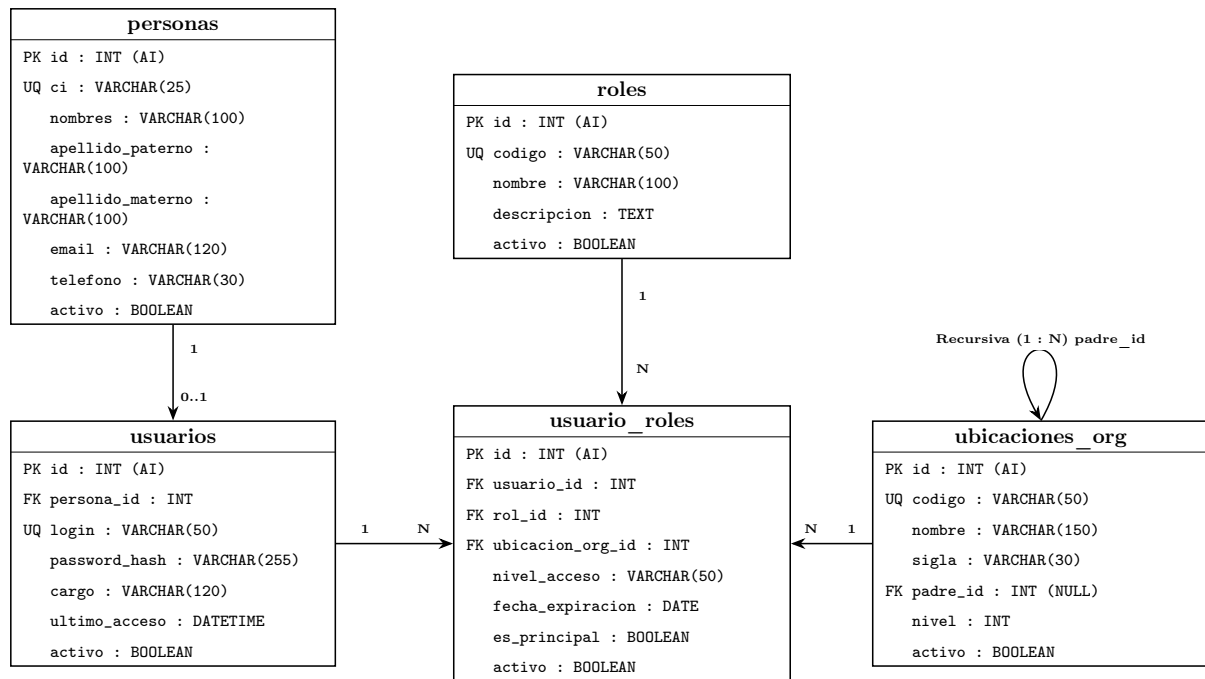


Figura 1: Diagrama Entidad-Relación (ERD) del Módulo de Identidad, Seguridad y Organigrama.

3. Sistema de Seguridad, Autenticación y Autorización

La seguridad del Sistema Wayka se diseñó bajo las pautas del estándar OWASP ASVS (*Application Security Verification Standard*), implementando un esquema desacoplado de autenticación basada en tokens no intrusivos en estado (*Stateless JWT*).

3.1. Ciclo de Vida y Flujo Criptográfico del Token JWT

El ciclo de autenticación opera de acuerdo con las siguientes fases:

- Recepción de Credenciales:** El cliente transmite login y password cifrados vía canal seguro HTTPS (TLS 1.3).
- Validación de Identidad y Hash:** El sistema consulta al usuario activo y realiza la comprobación computacional mediante la función unívoca `bcrypt.compare(password, password_hash)`. Si la comparación es infructuosa, se responde de inmediato con código 401 Unauthorized, sin divulgar si el error residió en el usuario o en la clave.
- Carga del Contexto de Rol Activo:** Se evalúan los roles vigentes en la tabla `usuario_roles`. Si el usuario cuenta con varios cargos, se activa automáticamente el rol designado como `es_principal = 1`, permitiendo conmutar a otros roles mediante el endpoint `/api/auth/switch-role`.
- Firma del Access Token y Refresh Token:** Se emiten dos artefactos criptográficos:

- **Access Token (Vigencia corta: 8 horas):** Contiene los *claims* de sesión requeridos por los middlewares de autorización sin requerir lecturas continuas a disco:

Claims = {userId, username, roleCode, ubicacionOrgId, nivelAcceso, iat, exp}

- **Refresh Token (Vigencia extendida: 7 días):** Permite al cliente solicitar renovación transparente de credenciales.
5. **Intercepción por Middleware:** Cada solicitud a rutas protegidas es interceptada por `authMiddleware.js`, extrayendo el encabezado **Authorization: Bearer <token>** y adjuntando la información decodificada al objeto `req.user`.

3.2. Matriz de Permisos Basada en Roles (RBAC)

El control de acceso gobierna las capacidades del sistema a través de la matriz de privilegios detallada en la Tabla 1.

Cuadro 1: Matriz de Control de Acceso Basada en Roles (RBAC) – Sprint 1

Módulo / Funcionalidad del Sistema	Admin Sistema	Admin Wayka	Ventanilla Única	Funcionario
Gestión de Usuarios y Credenciales	CRUD Total	× Bloqueado	× Bloqueado	× Bloqueado
Gestión de Personas (Padrón Base)	CRUD Total	× Bloqueado	× Bloqueado	× Bloqueado
Gestión de Organigrama y Unidades	CRUD Total	× Bloqueado	× Bloqueado	× Bloqueado
Configuración de Parámetros Globales	CRUD Total	× Bloqueado	× Bloqueado	× Bloqueado
Administración de Tipos de Trámite	× Bloqueado	Gestión Operativa Exclusivo	× Bloqueado	× Bloqueado
Anulación / Habilitación de Trámites	× Bloqueado	Exclusivo	× Bloqueado	× Bloqueado
Redirección de Procesos en Conflicto	× Bloqueado		× Bloqueado	× Bloqueado
Registro e Inicio de Trámites/VU	× Bloqueado	× Bloqueado	Creación / Emisión Operativo	× Bloqueado
Bloqueo / Desbloqueo Preventivo	× Bloqueado	× Bloqueado		× Bloqueado
Bandeja Virtual (Escritorio de Trabajo)	× Bloqueado	Supervisión	Acceso Completo Habilitado	Acceso Completo Habilitado
Atención de Trámites y Proveídos	× Bloqueado	× Bloqueado		
Derivación Libre y Retroceso	× Bloqueado	× Bloqueado	Habilitado	Habilitado
Carga de Adjuntos Digitales (PDF)	× Bloqueado	× Bloqueado	Habilitado	Habilitado
Consulta Pública de Hojas de Ruta	Permitido	Permitido	Permitido	Permitido
Cambio de Contraseña (PIN Seguro)	Propia	Propia	Propia	Propia

4. Especificación de la API (Endpoints Desarrollados)

La capa de API REST se implementó bajo el framework Express en Node.js, estructurando las respuestas en formato JSON uniforme compuesto por los campos {success, message, data}.

4.1. Catálogo de Rutas HTTP del Sprint 1

En la Tabla 2 se presenta el compendio de rutas disponibles en la versión 1.0.0-sprint1.

Cuadro 2: Endpoints Desarrollados y Expuestos en el Sprint 1

Método	Ruta de Acceso (Path)	Descripción Técnica	Nivel / Rol Mínimo
GET	/api/	Metadatos del microservicio y estado general.	Público
GET	/api/health	Verificación de salud y conectividad con MySQL.	Público
POST	/api/auth/login	Autenticación de usuarios y generación de JWT.	Público
GET	/api/auth/profile	Recupera datos de usuario y roles asignados.	Token Requerido
POST	/api/auth/switch-role	Conmuta el rol activo en sesión multi-rol.	Token Requerido
POST	/api/auth/change-password	Actualización de PIN con validación de complejidad.	Token Requerido

4.2. Ejemplos de Cargas Útiles (Payloads JSON)

4.2.1. 1. Inicio de Sesión Exitoso (POST /api/auth/login)

```

1 // Solicitud (Request)
2 {
3   "login": "admin",
4   "password": "admin123"
5 }
6
7 // Respuesta Exitosa (Response 200 OK)
8 {
9   "success": true,
10  "message": "Inicio de sesión exitoso",
11  "data": {
12    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
13    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
14    "user": {
15      "id": 1,
16      "username": "admin",
17      "nombres": "Administrador",
18      "apellidos": "Sistema Wayka",
19      "ci": "1000001",
20      "email": "admin@wayka.gob.bo",
21      "cargo": "Administrador Principal",
22      "activeRole": {
23        "usuario_rol_id": 1,
24        "rol_id": 1,
25        "rol_codigo": "ADMIN_SISTEMA",
26        "rol_nombre": "Administrador de Sistema",
27        "ubicacion_org_id": 6,
28        "ubicacion_codigo": "UNI-SIS",
29        "ubicacion_nombre": "Unidad de Tecnologías de Información y
30        Sistemas",
31        "nivel_acceso": "CONTROL_TOTAL",
32        "es_principal": 1

```



```
32     }
33   }
34 }
35 }
```

4.2.2. 2. Conmutación Dinámica de Rol (POST /api/auth/switch-role)

```
1 // Solicitud (Request)
2 {
3   "rolId": 2,
4   "ubicacionOrgId": 6
5 }
6
7 // Respuesta (Response 200 OK - Nuevo Token Re-firmado)
8 {
9   "success": true,
10  "message": "Rol activo cambiado a Administrador de Wayka",
11  "data": {
12    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.reSignedToken...",
13    "activeRole": {
14      "rol_id": 2,
15      "rol_codigo": "ADMIN_WAYKA",
16      "rol_nombre": "Administrador de Wayka",
17      "ubicacion_org_id": 6,
18      "ubicacion_nombre": "Unidad de Tecnologías de Información y
19                          Sistemas"
20    }
21  }
```

5. Pruebas y Validación (Quality Assurance)

La garantía de calidad del Sprint 1 se abordó mediante una batería de pruebas automatizadas que se ejecutan sin intervención manual en el entorno del servidor Node.js y MySQL.

5.1. Resultados del Banco de Pruebas de Autenticación (testAuth.js)

Se implementó un arnés de pruebas HTTP dinámico sobre Express para validar la robustez del subsistema criptográfico:

- **Test 1 (Login Válido):** Generación de token JWT, resolución de claims y asignación de rol principal por defecto (ADMIN_SISTEMA). → **Aprobado (200 OK)**.
- **Test 2 (Rechazo de Credenciales Inválidas):** Verificación de contraseñas erróneas y cuentas inexistentes. → **Aprobado (401 Unauthorized)**.

- **Test 3 (Protección de Rutas y Manipulación):** Acceso sin cabecera Authorization y con firma alterada. → **Aprobado (401 Denegado)**.
- **Test 4 (Conmutación Multi-Rol):** Verificación de actualización del token tras conmutar a rol ADMIN_WAYKA. → **Aprobado (200 OK)**.
- **Test 5 (Políticas de PIN Seguro):** Validación de reglas de complejidad (mínimo 6 caracteres, longitud máxima, combinación obligatoria de letras y números) y restauración de clave de prueba. → **Aprobado (400 Bad Request / 200 OK)**.
- **Test 6 (Cuentas Operativas Demo):** Verificación de inicio de sesión de Ventanilla Única (mfernandez) y Funcionario (cmamani). → **Aprobado (200 OK)**.

5.2. Resultados de Integridad del Modelo de Datos
(testDataModel.js)

La suite de persistencia evaluó los 11 modelos DAO con validación estricta de restricciones de negocio:

Cuadro 3: Resultados de la Validación de Integridad de Persistencia			
Módulo Evaluado	Regla de Negocio / Restricción Verificada	Resultado	Estado
Persona	Registro con CI unívoco y búsqueda por padrón civil.	Insert/Select OK	100 % Exitoso
UbicacionOrg	Cálculo recursivo de nivel jerárquico ($N_{padre} + 1$).	Nivel 2	100 %
Rol	Protección contra borrado con asignaciones vigentes.	Automático	Exitoso
Usuario	Generación de salt y hash bcrypt en creación.	Excepción capturada	100 %
UsuarioRol	Desmarque automático de otros roles principales.	Hash verificado	Exitoso
Correlativo	Generación secuencial atómica (TP-1/2026, TP-2/2026).	Exclusividad OK	100 %
Tramite	Creación en estado EN_ATENCION y movimiento inicial.	Cero colisiones	Exitoso
Movimiento	Inmutabilidad estricta: Rechazo de UPDATE y DELETE.	Orden 1 (INICIO)	100 %
SoftDelete	Bloqueo de eliminación de Persona con Usuario activo.	Mutación rechazada	Exitoso
		Integridad protegida	100 % Exitoso

6. Estado de Avance y Próximos Pasos

6.1. Tabla de Entregables Planificados vs. Ejecutados

El Sprint 1 ha cerrado formalmente con un **100 % de cumplimiento** de acuerdo al cronograma establecido en el documento de requerimientos maestros.

Cuadro 4: Balance de Cumplimiento del Sprint 1 (01 Sep – 14 Sep 2026)			
Objetivo Técnico / Tarea del Sprint	Planificado	Completado	Observaciones / Estado
Configuración de entorno Node.js, Express y MySQL 8.0+	100 %	100 %	Entorno de desarrollo Docker/Local operativo.
Esquema Relacional unificado y 11 scripts de migración	100 %	100 %	Migraciones versionadas mediante <code>migrator.js</code> .
Módulo Criptográfico y Autenticación JWT Stateless	100 %	100 %	Sockets cerrados, Bearer token y claims de rol.
Matriz RBAC y Middleware de Autorización por Rol	100 %	100 %	Protección granular en API.
Carga de Semillas Base (Roles, Organigrama, Cuentas Demo)	100 %	100 %	3 usuarios base y 6 unidades del organigrama.
Modelos DAO para Entidades Base (CRUD y Soft-delete)	100 %	100 %	Lógica de negocio e integridad en JavaScript.